Project

Title: Phone directory

Course: CSE103

Section: 13

Group members:

- Rifa Azad 2022-1-60-265
- Sanjida Islam Aksa 2022-1-60-254

Report:

Problem Statement:

This project is on a simple telephone directory to store data and information using array of structure, file handling, loop and condition and others, where there 2 types of data character(mainly strings) and integer.

Feature Discussion:

In the C program, we used structure using **struct** keyword to define variable type; used file to store data and used different function to perform many other actions on those data. Our 1st thought to store data by creating a file and later on create other functions to perform various actions. Given box is the header file along with structure declaration.

```
//header files

#include<stdio.h>

#include<string.h>

#include<stdlib.h>//needed for exit()

#define MAX 11

/* Simple Phone directory */

//these define is used so that in future if needed

//can change the sizes easily rather than finding in all over the code

typedef struct phonebook

{//done

    int no;

    char name[40];

    char number[11];//number would be 10 digits

    char mail[30];

    int code;

} book; //with this can declare in functions
```

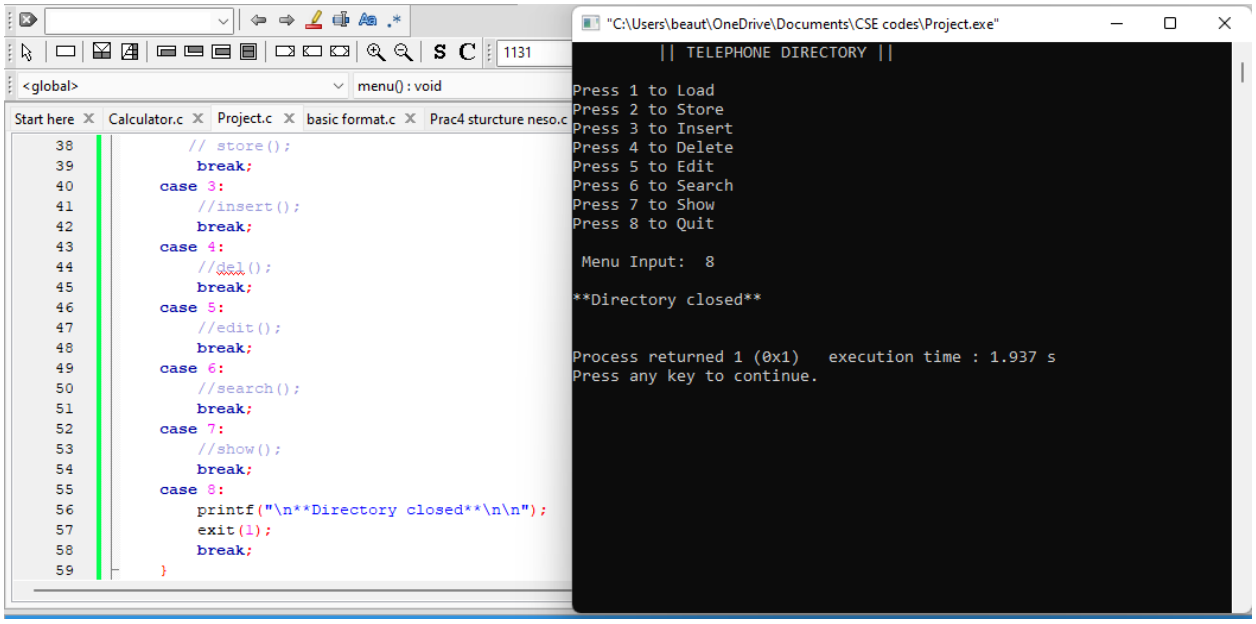
Then had a **main()** program will start. Like given below,

```
int main(void)

{ //all implentation will be done in various functions separatley

    // that's why main() does not have much content

    printf("\n\t || TELEPHONE DIRECTORY || \n\n");

    menu();

    return 0;
```

```
void menu()
{
    int entry;
    while(1)
    {
        printf("\n");
        printf("Press 1 to Load\n");
        printf("Press 2 to Store\n");
        printf("Press 3 to Insert\n");
        printf("Press 4 to Delete\n");
        printf("Press 5 to Edit\n");
        printf("Press 6 to Search\n");
        printf("Press 7 to Show\n");
        printf("Press 8 to Quit\n");
        printf("\nMenu Input:  ");
        scanf("%d", &entry);
    switch(entry)
{
     //depending on entry, functions will be called
        case 1:
            load();
            break;
        case 2:
            store();
            break;
        case 3:
            Insert();
            break;
        case 4:
            Delete();
            break;
        case 5:
            edit();
            break;
        case 6:
            search();
            break;
        case 7:
            show();
            break;
        case 8:
            printf("\n***Directory closed***\n\n");
            exit(1);
            break;
        }
//end of switch
    }
}//end of function
```

In **main()** a function was called named **menu()** which shows different options and has codes to call other functions.

Each entry from Keyboard (numbers only) will go to different switch case and will call designated functions. This switch case was inserted in a loop so that user can call different function until the work is finished and close it by calling for 8 through keyboard like the below picture**:**



.

Now, discussion on used functionalities:

Store:

It was the 1st function to be implementing in our C program, as need to have data stored in a file to run other functions. We used **store()** function for this.

While creating this function the file .txt was in w+ mode thus "***DIRECTORY***" was created and entered the loop to take values for each people variable.

In this function, user was asked to enter the amount of values they want to store in the phonebook that's why rather than an array a pointer variable was define as structure type as it was unknown at first that how many entries will take place and depending on that for loop was written.

```
void store()
{//done
    book *p;//as don't know the amount of input
    int n, i;

    FILE *fp=NULL;
    fp = fopen("DIRECTORY.txt", "r+");
    if (fp == NULL)
    {
        printf("!!Something went wrong!!\n");
        exit (1);//terminates program
    }

    printf("Enter the amount of input: ");
    scanf("%d", &n);

    p = (book*)calloc(n, sizeof(book));
//calloc used to allocated memory of *p, as it has multiple blocks
for (i=0; i<n; i++)
    {
        p[i].no = i ;
        printf("\nEnter name: ");
        fflush(stdin);//to clear/flush the buffer of output.
        gets(p[i].name);

        printf("Enter telephone number: ");
        fflush(stdin);
        gets(p[i].number);
        printf("Enter mail address: ");
        fflush(stdin);
        gets(p[i].mail);
        printf("Enter area code: ");
        fflush(stdin);
        scanf("%d", &p[i].code);

//printing the inputs in the file
        fprintf(fp, "%d %s %s %s %d\n", p[i+1].no, p[i].name, p[i].number, p[i].mail, p[i].code);
    }
    printf("\n You have entered %d contact details.\n", i);
    fclose(fp);
}//end of function
```

(Uses of **calloc()** function will be described later on)

fflush(stdin) is used to clear/flush the buffer of output.

After running the code along with this function and storing data in the file, the mode was changed to r+ as in w+ the file's previous contents will be erased if the program was build and run again.

Limitation of this function:

- In this function in a+ mode, if the program is ran again the cursor will be at the end of the file and will add inputs but the serial will start from 0. If used r+ mode then all new inputs will overwrite the previous contents as the cursor

will be at the beginning of the file when the file will be opened again.

- But in r+ mode file will not be created if not exists, in a+ mode will create one.
- Even it creates a file in a+ mode but may not have the same data that I inputted or inserted.

Benefits of this function:

- Whoever has this code can create their own phonebook with any amount of entries (might need to change MAX in define) by just changing the function's file mode in w mode.

❖ Some things are common in each function even though they have different implantation and each are connected with others (will be discussed later on), such as

```
if (fp == NULL)
   {
printf("!!Something
went wrong!!\n");
      exit (1);
   }
```

This condition was used in every section where a file has been opened.
This condtion shows if the file has been opened successfully or not. If the condition is true that means the file pointer is holding a null value which shows the written input and let the user know that the file is not opened successfully.

Also, every time a file has been opened it was closed after completing operations.

```
void load()
{//done
    book p;


    FILE *fp=NULL, *fp2=NULL;
    char ch;
    fp = fopen("DIRECTORY.txt", "r");
    if (fp == NULL)
    {
        printf("!!Something went wrong!!\n");
        exit (1);
    }


    fp2 = fopen("Copy of DIRECTORY.txt", "w+");
    if (fp2 == NULL)
    {
        printf("!!Something went wrong!!\n");
        exit (1);
    }


    while((ch = fgetc(fp))!= EOF)
        putc(ch, fp2);


    fclose(fp);
    fclose(fp2);
    printf("\n");
    printf("\t*Loading DONE*\n");
}//end of function
```

Load:

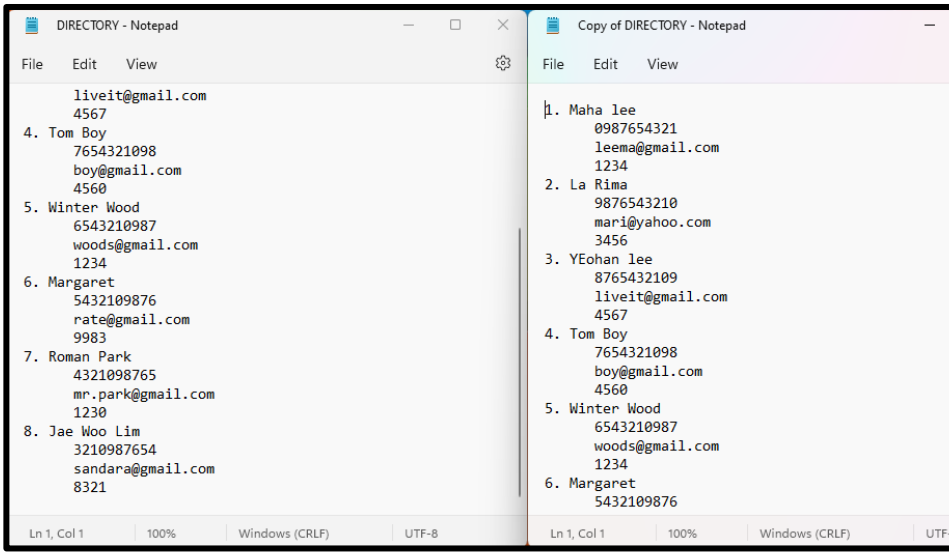
Load means to load the data from the current file system; here it is ***"DIRECTORY".*** So, loaded all its data to a different file; to put it another way copied its content in another file. Data was copied character by character till it the cursor reached at the end of file.

It could have read the data as strings using **fgets()** but as data is of different type also different bytes, so specifying them separately is little bit twisting so better to walk on safe path reading char by char.

For this, needed to open the copy version in w+ mode so that it can write the data from the file also read if necessary. Through the copy version of the file we can check if the file was loaded successfully or not that's why copied it another file.

Below picture is after storing and loading the file of the files. (This was the 2nd initialed pattern how wanted to store the data but faced problems later for other function so changed it to another pattern, final pattern has been provided with the whole file in submit section)

```
void show()
{
    //done
    book p;
    char ch;

    FILE *fp = NULL;
    fp = fopen("DIRECTORY.txt", "r");
    if (fp == NULL)
    {
        printf("!!Something went wrong!!\n");
        exit (1);
    }

    printf("\n");
    while((ch=fgetc(fp)) != EOF)
    {
        putchar(ch);
    }
fclose(fp);
}//end of function
```



Show:

This function is used to show everything that is present in the file. Here, file was opened in r mode as it will just read data and print it in console window.

It will read from beginning to the end of file character by character and print each character according to the file.

- ***Limitations of this function:***

This function can't read the file by strings.

- *Improvements that can be add in this function:*

This function can be in execute by reading string(tried numerous times but

always ended up messing up in outputs, that's why kept the code by strings in comment and showed char by char, here showing only the code that ran successfully; not the string one).

[Thoughts while doing this project are written in () and problem occurrence and reasoning is written with document bullet(4doted one)]

- Program may have some errors as whenever tried to work with structure variable it wasn't showing elements properly not all cases, tried different ways, then ended up using typedef but still same problem again and again which why in our program has some fault which will be discussed in occurred functions.

- This is why used **calloc()**. Thought maybe file has data somehow could not trace it back so created a memory for the array in heap but still didn't work the way wanted (discussed in occurred functions) and give desire result. But if go through the code it is written correctly.

To continue with the discussion of functionalities:

```
void Insert()
{//done
    book p[MAX];
    char ch;
    int serial, count_line=1;
    FILE *fp;
    fp = fopen("DIRECTORY.txt", "a+");
    if (fp == NULL)
    {
        printf("!!Something went wrong!!\n");
        exit (1);//terminates program
    }
    while((ch=fgetc(fp))!= EOF)
    {
        if(ch == '\n')
            count_line++;
    }
    serial = count_line-1;
//lines are initialized 1 as even the cursor is at the begining of the files show 1 line
    for (int i=0; ; i++)
    {
        if(serial==MAX)
        {
            printf("\n\t!!!Directory Full!!! Delete some data to insert more\n\n");
            break;
        }
        serial++;
        printf("%d.", serial);
        printf("\nEnter name: ");
        fflush(stdin);
        gets(p[i].name);

        printf("Enter telephone number: ");
        fflush(stdin);
        gets(p[i].number);

        printf("Enter mail address: ");
        fflush(stdin);
        gets(p[i].mail);

        printf("Enter area code: ");
        fflush(stdin);
        scanf("%d", &p[i].code);

        //printing the inputs in the file
        fprintf(fp, "%d %s %s %s %d\n", serial, p[i].name, p[i].number, p[i].mail, p[i].code);
        printf("\n\tInsertion Complete.\n");
        break;

    }
    fclose(fp);
}//function end
```

Insert:

This function is used to insert new data along with the previous contents.

Here, need to open the file directory in a+ mode as then the cursor will be at the end of the file and new inputs will be stored accordingly.

After giving the inputs operation will be complete with a

message printed on console window then if go to **show()** then will show all the inputs of the file both previous ones and new ones. As the whole **switch()** is in loop so user can input multiple inputs but if inputs are reached MAX then will stop inputting with a message on console window thus **Delete()** function comes.

Limitations of this function:

- If want to insert a certain value like only code or e-mail then have to write from start like- name , phone number then e-mail and code. So it is lcumbersome.

Delete:

If want to delete a certain element or data from file then this function will be called to execute the program to delete.

Here, after opening the file **"*DIRECTORY*",** a temporary file has been created. So that after executing the code a file will be set without the deleted data.

After opening the file, **fread()** function will read the fp file if inputted data is found then will put 1 in **found** variable else will keep writing in the temporary file.

If found the inputted data then will not enter else condition so inputted data will not be in temporary file. After reading the whole "*DIRECTORY*" file will close both files and reopen both to copy temporary file's data to "*DIRECTORY*" file.

Hence, in **"*DIRECTORY*"** file will have data without the inputted data. In a nut shell, inputted data is deleted.

This function call is similar to **edit()** , dissimilarity is discussed in **edit()** .

Limitations of this function:

- If wanted to delete a certain data of a serial no then it would not have been possible with this function.
  8 Kaisar Ali 890321 ali@yahoo.com 781
  If wanted to delete only email, then had to delete this whole content.

- This function does not work properly if called. Could not find the reason, if go by step by step then seems like correct but does not work, could not find any more idea so putted the code in project after trying separately many times.

```
void Delete()
{
   book p1;
   char ch;
   int n, serial, found=0, look ;
   FILE *fp=NULL, *fp1=NULL;
   fp = fopen("DIRECTORY.txt", "r");
   if (fp == NULL)
   {
      printf("!!Something went wrong!!\n");
      exit (1);
   }
   fp1 = fopen("TEMPORARY.txt", "r");
   if (fp1 == NULL)
   {
      printf("!!Something went wrong!!\n");
      exit (1);
   }
   printf("\nEnter the serial want to
delete: ");
   scanf("%d", &look);
   //printf("%s\n, p.name");
   while(fread(&p1, sizeof(book), 1, fp))
   {
      if(p1.no == look)
         found = 1;
      else
         fwrite(&p1, sizeof(book), 1, fp1);
   }fclose(fp);
   fclose(fp1);
if(found == 0)
      fp1 =
fopen("TEMPORARY.txt",
"r");
   fp =
fopen("DIRECTORY.txt",
"w+");

   while(fread(&p1,
sizeof(book), 1, fp1))
      fwrite(&p1, sizeof(book),
1, fp);

   fclose(fp);
   fclose(fp1);
}//function end
```

❖ Both of us (group members) tried with this function in different way. I am inputting those too in report (at the end of the report file; to show the efforts we made)

❖ What I think the problem is maybe the buffer has some input which creates problem with running the code and my function is unable to solve this.

```
void search()
{
    book p;
    char ch, look[100];
    int n, serial, found=0, count_line =1;
//lines are initialized 1 as even the cursor is at the begining of the
//files show 1 line
    FILE *fp=NULL;
    fp = fopen("DIRECTORY.txt", "r");
    if (fp == NULL)
    {
        printf("!!Something went wrong!!\n");
        exit (1);
    }

    while((ch=fgetc(fp))!= EOF)
    {
        if(ch == '\n')
            count_line++;
    }
    serial = count_line-1;

    /*fseek(fp, 0, SEEK_END);
    n = ftell(fp)/sizeof(book);
    //to get the number of record saved but didn't work*/
printf("\n\t**You have %d data saved.**\n", serial);
    printf("\nEnter the serial want to search: ");
    scanf("%d", look);

    //printf("%s\n, p.name"); shows weird value
    while(fread(&p, sizeof(book), 1, fp))
    {
        if(p.no == look)
        {
            found = 1;
            printf("%d %s %s %s %d",p.no, p.name, p.number, p.mail, p.code);
        }
    }
    if(found == 0)
        printf("\n!! NOT FOUND!!\n");
    /*char line[100];
    while(fgets(line, sizeof(line), fp))
    {
        if(strcmp(line, look)==0)
        {
            found = 1;
            printf("%s\n",line);
        }
    }
    if(found == 0)
        printf("\n!! NOT FOUND!!\n");*/

    fclose(fp);
}//function end
```

Search:

This function is to search data from file. In this project most diverse function is this one. This function can have many types of codes and functions and have function within. (Unfortunately codes in our program didn't run properly / didn't show outputs the way I wanted)

At first it will count line of the file's data show it as how many of data are saved. (Used this to work on **for** loop's condition but later on deleted that as outputs were not showing).

This function will read the file and put it in an

if condition with the input value to check if that exists or not. If found will print it's all element of the structure and if not then will print a message on console window to declare that data has not been found.

- As it didn't work so had to change the pattern of storing data (mentioned before in Load with pictures of files) and changed to current one, as thought maybe the way input was taken created problem.

Limitations of this function:

- It can only search with numeric value here it is serial number. If serial no is not known then will not be able the designated data if there is a lot of data in the phone book.

Improvements that can be done on this function:

- It can have more functions where the statements will be to search data with any of the structured element, in our program it can be name or number or mail or area code.
- For function to search with name it can have two types; one with first name, another with last name.
- These functions can be modified as, if there are people with same name or surname both of their information will show on console window. If there is people's information that has same area code then there information will come together. People having same operator can be found with modified search function.
- If a name or string in input is similar to a data in the file then a condition code can show suggested information like how it works in different search engine.

- Wanted to implement 1st three of these suggestions, also had idea to how but it wasn't showing input however I change the function's code, so didn't try to implement the improved ideas.
- Tried to convert file's data to structure and then implement the **search()** but conversion did not work.

Edit:

This function is called to edit a certain data in the file. This function is mixed version of **search()**, **insert(),** also **Delete()** is form by using some of its parts.

As need to find the data first which need to be edited so same like **search()** will read the file first then look for the input in file, also write it in another temporary file opened with the "***DIRECTORY***" file then will enter **if condition** by checking if input and serial number(searching through serial) matched or not .

If condition is true then will take input for that particular lines data and this new input will also add in temporary file as after coming out of **if condition,** it will face **fwrite()** .

If did not find the value then will print message in console and write temporary file contents in ***"DIRECTORY"*** file. So no changes in ***"DIRECTORY"*** file.

After all this operations both files will be close.

Limitations of this function:

- If the code had ran successfully, then to edit a certain specific data needed to write whole data again, such as
  8 Kaisar Ali 890321 ali@yahoo.com 781
  if wanted to edit just this value's area code then had to write both name, email and number just to change the code, which is cumbersome.

Improvements that can be add in this function:

- Can make functions to edit a lines certain content like,
  8 Kaisar Ali 890321 ali@yahoo.com 781
  In this line will only edit any of these elements, 1 or 2. And write it back to it's desire place.
- Can create function to change position such as above line is in 8no line but can put it in 4th line.

  (function code given below)

```
void edit()
{
    book p1;
    char ch;
    int n, found=0, look;

    FILE *fp=NULL,
*fp1=NULL;
    fp =
fopen("DIRECTORY.txt", "r");
    if (fp == NULL)
    {
        printf("!!Something went
wrong!!\n");
        exit (1);
    }

    fp1 =
fopen("TEMPORARY.txt",
"r");
    if (fp1 == NULL)
    {
        printf("!!Something went
wrong!!\n");
        exit (1);
    }

    printf("\nEnter the serial
want to edit: ");
    scanf("%d", &look);//4 no
input want to edit

    //printf("%s\n, p.name");
while(fread(&p1, sizeof(book), 1, fp))
    {
        if(p1.no == look)
        {
            found = 1;//found 4 no input
            //if ntot then writing in temp file
                //taking inputs for 4no output
                printf("%d.", look);
                printf("\nEnter new name: ");
                fflush(stdin);
                gets(p1.name);
printf("Enter new telephone number:
");
                fflush(stdin);
                gets(p1.number);
        printf("Enter  new mail address: ");
                fflush(stdin);
                gets(p1.mail);
        printf("Enter new area code");
                fflush(stdin);
                scanf("%d", &p1.code);
                //printing the inputs in the file
  fprintf(fp, "%d %s %s %s %d\n", look,
p1.name, p1.number, p1.mail,
p1.code); //new inputs added in 4no
serial }//end of if condition
/*until not found then will print those
values in fp1*/
  fwrite(&p1, sizeof(book), 1, fp1);
    }
    fclose(fp);
    fclose(fp1);
if(found == 0){
        fp1 =
fopen("TEMPORARY.txt",
"r");
    fp =
fopen("DIRECTORY.txt",
"w+");
    while(fread(&p1,
sizeof(book), 1, fp1))
        fwrite(&p1,
sizeof(book), 1, fp);
    }

    fclose(fp);
    fclose(fp1);

}//end of function
```

- If read line by line of this function then it seems to work but not in program, does not show any output for inputs that are in files.

- As both edit and delete are related to **search()** (implemented **search()** code to these functions) and **search()** did not work properly so initial thinking is that maybe that's why **edit() Delete()** didn't work.

This was the discussion on every function and formats of this project along with comments and thoughts. Limitations for each function have been written in their designated parts along with future plans.

Limitations for this whole project:

- Functions that did not work are the only limitation that is stopping the program to run smoothly or else it was executed nicely.
- Some other ways that we tried to do for those function (which did not run properly to work is given below:

```
void Delete()
{
    char namee[MAX];
    printf("Enter the name of the person you want to remove from telephone directory : ");//a print function for the name of the person we want to remove from telephone directory
    getchar();
    scanf("%[^\n]s",namee);
    printf("%s",namee);
    FILE *fp,*temp;
    fp = fopen("phonebook_db.txt", "r");
    temp = fopen("temp","w+");
    if (fp == NULL)   //file pointer returns null that’s mean we are unable to open the file
    {
        printf("FIle Not Found.\n");
        printf("Press any key to continue....\n");
        return;
    }
else
    {
        people p;
        int flag = 0;
        while (fread(&p, sizeof(p), 1, fp) == 1)
        {
            int result= strcmp(&p.name, &namee);
            if(result==0)
            {
                printf("Person removed successfully..!\n");
                flag = 1;
            }
            else fwrite(&p,sizeof(p),1,temp);
            fflush(stdin);
        }
        if(flag == 0)
        {
            printf("No record found for %s member.\n",namee);
        }
      fclose(fp);  //fclose function for close this file
        fclose(temp);

remove("phonebook_db.txt");

rename("temp","phonebook_db.txt");
        fflush(stdin);
        printf("Press any key to continue....\n");
    }
}//end of function
```

```
void Edit()

{

    char namee[MAX];

    printf("Enter the name of the person you want to Edit from telephone directory : ");

    getchar();

    scanf("%[^\n]s",namee);

    printf("%s",namee);

    FILE *fp,*temp;

    fp = fopen("phonebook_db.txt", "r");

    temp = fopen("temp","w+");

    if (fp == NULL)

    {

        printf("FIle Not Found\n");

        printf("Press any key to continue....\n");

        return 0;

    }

else {

        int flag = 0;

        people p;

 while (fread(&p, sizeof(p), 1, fp) == 1)

        {

            int result= strcmp(&p.name, &namee);

            if(result==0)

            {

                getchar();

                printf("Enter Name : ");

                scanf("%s",&p.name);

                printf("Enter Area Code : ");

scanf("%d",&p.area_code);

                printf("Enter Address : ");

                scanf("%s",&p.address);

                printf("Enter Email : ");

                scanf("%s",&p.mail);

                fwrite(&p, sizeof(p), 1, temp);

                fflush(stdin);

                printf("Record added Successfully.\n");

                flag = 1;

            }

            else fwrite(&p,sizeof(p),1,temp);

            fflush(stdin);

        }

if(flag == 0)

        {

            printf("No record found for %s .\n",namee);

        }

        fclose(fp);//fclose function for close this file

        fclose(temp);

remove("phonebook_db.txt");

rename("temp","phonebook_db.txt");

        fflush(stdin);

        printf("Press any key to continue....\n");

    }

}//end of function
```